



SafeDoc
Automation



SafeDoc
Automation

Automation
SafeDoc

I.E.S. CLARA DEL REY

Comunidad de Madrid · Departamento de Informática y Comunicaciones



MEMORIA FINAL DE PROYECTO

SafeDoc Automation

Plataforma de Automatización y Seguridad Documental para PYMEs

CICLO FORMATIVO DE GRADO SUPERIOR

Desarrollo de Aplicaciones Web (DAW)

MÓDULOS IMPLICADOS

DWES (0613) · DWEC (0612) · DAW Proyecto (0616) · EIE · Sostenibilidad

Autora	Ana Cifuentes Romero — ana.cifuentes@educa.madrid.org
Empresa colaboradora	Amigo Informático Europa, S.L.
Tutor de empresa	Héctor Caparrós Barral
Tutora del centro	Mercedes Rodríguez Villafáfila
Convenio	Nº 2454 — Anexo 337
Curso académico	2025/2026 · Madrid, 28 de mayo de 2026

El IES Clara del Rey queda licenciado para el uso docente y de investigación, sin ánimo de lucro, del presente trabajo y sus métodos, productos y resultados, con mención explícita de su autora.

1. Índice de Contenidos

1. Índice de contenidos.....	3
2. Resumen ejecutivo.....	3
3. Contexto profesional y cliente	5
3.1. La empresa: Amigo Informático Europa, S.L.....	5
3.2. El cliente: Gestoría Linares & Asociados, S.L.P.....	5
4. Antecedentes y justificación	6
4.1. Situación de partida.....	6
4.2. Análisis del sector y posicionamiento	6
4.3. Justificación curricular.....	6
5. Objetivos.....	8
5.1. Objetivo general	8
5.2. Objetivos específicos.....	8
6. Análisis: Requisitos y Casos de Uso	9
6.1. Enfoque metodológico en la captura de requisitos.....	9
6.2. Requisitos funcionales	9
6.3. Requisitos No funcionales	9
6.4. Casos de Uso principales.....	10
6.5. Trazabilidad: Del análisis al código	10
7. Diseño y Arquitectura	12
7.1. Visión general de la arquitectura.....	12
7.2. Flujo de comunicación entre capas.....	12
7.3. Modelo de datos	12
7.4. Diseño de la interfaz.....	13

8. Planificación y metodología.....	14
8.1. Metodología ágil	14
8.2. Cronograma	14
8.3. Control de versiones.....	14
9. Desarrollo técnico	16
9.1. Marco Teórico: Estado del Arte	16
9.1.1. Automatización documental e integradores iPaaS	16
9.1.2. Librerías Java para procesamiento binario.....	16
9.1.3. Integridad documental con SHA-256.....	16
9.2. Capa PHP / Laravel / Filament	16
9.2.1. Estructura del proyecto	16
9.2.2. Procesamiento asíncrono con Jobs	16
9.2.3. Seguridad y autenticación.....	17
9.3. Microservicio Java con Spring Boot	17
9.3.1. Arquitectura del microservicio.....	17
9.3.2. Procesamiento Excel con Apache POI	17
9.3.3. Generación y extracción de PDF con Apache PDFBox	18
9.4. Worker Node.js + Puppeteer	18
9.5. Integración continua.....	18
10. Sostenibilidad e impacto	19
10.1. Impacto ambiental.....	19
10.2. Impacto social y económico	19
11. Empresa, mercado y modelo de negocio.....	20
11.1. Posicionamiento estratégico	20
11.2. Análisis de la competencia	20
11.3. Modelo de Monetización.....	20

- 11.4. Indicadores estratégicos (KPIs) 20
- 11.5. Riesgos y mitigaciones 20
- 12. Resultados y pruebas..... 22
 - 12.1. Plan de pruebas 22
 - 12.2. Métricas de rendimiento..... 22
 - 12.3. Entregables del proyecto..... 23
- 13. Conclusiones 24
 - 13.1. Logros alcanzados 24
 - 13.2. Aprendizajes personales 24
 - 13.3. Líneas futuras 24
- Bibliografía y Referencias 25
- Anexo A: Esquema de Base de Datos SQL 26
- Anexo B: Stack tecnológico completo..... 28
- Anexo C: Glosario de Términos 29

1. Resumen ejecutivo

SafeDoc es una aplicación web orientada a la transformación digital de pequeñas y medianas empresas que gestionan grandes volúmenes de documentación (contratos, informes, registros administrativos). El sistema automatiza tareas repetitivas de procesamiento de archivos Word, Excel y PDF, genera versiones protegidas e inmutables de los documentos originales y proporciona un sistema de auditoría basado en funciones hash SHA-256.

SafeDoc Automation es una plataforma web desarrollada durante el periodo de Formación en Centros de Trabajo en Amigo Informático Europa, S.L., como encargo de Gestoría Linares & Asociados, S.L.P. Su objetivo es automatizar la generación masiva de documentación administrativa —nóminas, facturas, modelos AEAT, contratos laborales— a partir de plantillas reutilizables, así como la extracción y validación de datos contenidos en archivos recibidos de terceros.

Trabajar con tres entornos de desarrollo diferentes ha sido una de las decisiones más importantes del proyecto: un núcleo web construido en PHP sobre Laravel 11 con el panel administrativo Filament 3 y el stack TALL (Tailwind, Alpine.js, Laravel, Livewire); un microservicio Java con Spring Boot que asume el procesamiento masivo de archivos Excel y PDF mediante Apache POI y Apache PDFBox; y un worker Node.js con Puppeteer que automatiza la extracción de datos desde fuentes externas como la sede electrónica de la Agencia Tributaria.

Los resultados esperados son: reducir en al menos un 60 % el tiempo medio dedicado a tareas documentales por empleado del despacho, bajar la tasa de errores de copiado por debajo del 0,5 %, procesar lotes de hasta 500 documentos en menos de cinco minutos sin bloquear la interfaz, garantizar la integridad de los archivos originales mediante el principio de inmutabilidad, implementar seguridad documental real (cifrado AES-256, restricción de permisos), registrar toda actividad sobre los documentos para facilitar la trazabilidad y el control y sentar las bases de un producto comercializable para despachos de gestoría de menos de 20 empleados.

Lo que realmente hace diferente a este proyecto radica en su diseño arquitectónico: Haber separado el motor de procesamiento en Java del servidor web principal en PHP. En lugar de sobrecargar a Laravel con tareas síncronas pesadas —como procesar archivos de grandes dimensiones en documentos Word, Excel o PDF—, se optó por aislar esta lógica en un microservicio independiente. Esta estrategia no solo optimiza el rendimiento general y evita caídas por saturación de memoria en el servidor web, sino que permite escalar el componente de procesamiento de forma autónoma según las necesidades reales de carga del despacho. Desde la perspectiva académica, este enfoque me ha permitido consolidar de manera práctica los resultados de aprendizaje clave de los módulos de Programación y Desarrollo Web en Entorno Servidor (DWES), logrando que dos sistemas completamente distintos trabajen juntos en producción, demuestra una integración real.

3. Contexto profesional y cliente

3.1. La empresa: Amigo Informático Europa, S.L.



Amigo Informático Europa, S.L. es una PYME madrileña especializada en desarrollo de software a medida y servicios de soporte técnico para empresas de servicios profesionales. Su modelo combina proyectos llave en mano (es un servicio en el que un proveedor se encarga de todo el proceso de creación de un producto, desde la idea hasta su puesta en marcha, para entregárselo al usuario final 100% operativo y listo para producir.), con contratos de mantenimiento recurrente, lo que le proporciona una cartera estable de clientes en los sectores legal, sanitario y de distribución.

La empresa apuesta por tecnologías abiertas: PHP/Laravel para desarrollos web y Java para procesos de back-office de alto rendimiento. La adopción reciente del stack TALL y del panel Filament 3 le ha permitido reducir significativamente los plazos de entrega de paneles administrativos sin sacrificar calidad ni accesibilidad.

3.2. El cliente: Gestoría Linares & Asociados, S.L.P.



Gestoría Linares & Asociados, S.L.P. es un despacho fiscal-laboral con sede en el barrio de Salamanca (Madrid), con doce empleados y aproximadamente 320 clientes activos —principalmente autónomos, micropymes y pequeñas empresas del sector servicios—. Es cliente recurrente de Amigo Informático Europa desde marzo de 2022.

Durante el último trimestre de 2025 el despacho cuantificó su problema: genera mensualmente alrededor de 3.200 documentos (nóminas, facturas, modelos AEAT 303/130/111/115, contratos y certificados de retenciones) con un tiempo medio de 4-6 minutos por documento y una tasa de errores de copiado cercana al 3 %, lo que obliga a reemitir aproximadamente 100 documentos cada

mes. Una estimación interna sitúa en 110-130 horas mensuales el trabajo administrativo liberalizable mediante automatización, equivalente a la jornada de un empleado a tiempo completo.

4. Antecedentes y justificación

4.1. Situación de partida

El proceso documental actual del cliente presenta tres limitaciones estructurales que justifican la inversión en una solución a medida:

- Reescritura manual de datos entre Excel, Word y PDF, que multiplica los puntos de error y dificulta la trazabilidad de cualquier modificación posterior.
- Ausencia de control de versiones sobre plantillas documentales, con copias dispersas en carpetas locales y de red que provocan el envío ocasional de versiones obsoletas a clientes finales.
- Desconexión entre la generación documental y el flujo de firma electrónica, que obliga a descargar, firmar y volver a subir cada documento de forma individual.

4.2. Análisis del sector y posicionamiento

La gestión documental en las PYMEs españolas sigue siendo, en gran medida, un proceso artesanal y propenso a errores. Según el Instituto Nacional de Estadística, más del 70% de las empresas con menos de 50 empleados carece de herramientas automatizadas para el tratamiento de documentos internos. El trabajo diario con procesamiento de textos Word, hojas de cálculo Excel y archivos PDF implica tareas repetitivas como: aplicar contraseñas de apertura y edición, restringir permisos de impresión y copia, generar versiones de solo lectura, y verificar que los archivos enviados a terceros no han sido alterados.

Estas tareas se realizan manualmente, una a una, lo que genera cuellos de botella en departamentos administrativos y aumenta el riesgo de errores: archivos enviados sin proteger, versiones duplicadas sin control o documentos modificados accidentalmente.

En el caso que nos ocupa, España cuenta con cerca de 80.000 despachos de gestoría, asesoría y consultoría profesional según el Consejo General de Colegios de Gestores Administrativos. Se trata de un tejido empresarial muy fragmentado, dominado por pequeñas estructuras de menos de quince

empleados, que en los últimos años han acelerado su digitalización, forzadas por iniciativas regulatorias como FACTURAE, el Suministro Inmediato de Información (SII) o el reciente sistema Verifactu.

El mercado de software para despachos profesionales está dominado por grandes suites integradas como A3 (Wolters Kluwer), Sage Despachos Connected o Holded. Estas plataformas ofrecen soluciones tan masivas y genéricas que presentan tres inconvenientes críticos para las pequeñas gestorías:

- **Barrera económica y rigidez:** imponen un modelo de licencia por usuario altamente costoso y módulos cerrados que obligan a pagar por funcionalidades que el pequeño despacho no requiere.
- **Incompatibilidad y silos de información:** operan como sistemas cerrados con bases de datos propietarias, bloqueando la capacidad de integrar flujos externos de automatización masiva.
- **Falta de procesamiento pesado flexible:** carecen de motores dedicados que permitan procesar cientos de estructuras binarias en una sola pasada y aplicar cálculos de integridad asíncronos sin degradar la interfaz web.

Por otro lado, soluciones de firma y gestión documental genéricas como Adobe Acrobat Pro, PandaDoc o DocuSign destacan por su robustez legal, pero ignoran las necesidades particulares y los flujos de trabajo reales de este sector: no validan datos contra la base de datos interna de la gestoría ni automatizan la extracción desde la sede electrónica de la Agencia Tributaria.

SafeDoc Automation se posiciona deliberadamente en el nicho intermedio: no aspira a ser una suite contable completa, sino una herramienta de automatización modular, ligera, integrable mediante API con la suite que ya utilice el cliente y, sobre todo, asequible para despachos de menos de veinte empleados.

4.3. Justificación curricular

El proyecto cubre de forma directa los siguientes resultados de aprendizaje del ciclo DAW:

Módulo	RA cubierto	Cómo se cubre
DWES (0613)	RA1: Desarrollo de arquitecturas desacopladas	Separación de responsabilidades: el servidor web corre en Laravel 11 y el motor pesado se aísla en un microservicio de Spring Boot.
DWES (0613)	RA6 — Acceso seguro a datos	ORM Eloquent con parámetros enlazados, autenticación Laravel Sanctum, políticas de autorización por rol.
DWES (0613)	RA8 — Páginas dinámicas servidor	Jobs de Laravel para procesamiento asíncrono, API REST documentada con Scribe.
DWEC (0612)	RA1 — Comunicación asíncrona	Livewire para actualizaciones reactivas, Alpine.js para interactividad cliente, polling de estado de jobs.
DWEC (0612)	RA6 — Interfaces accesibles	Stack TALL con Filament 3; conformidad WCAG 2.1 AA verificada con axe-core.
DAW Proyecto (0616)	RA2 — Diseño del proyecto	Análisis de requisitos RF/RNF, casos de uso UML, modelo de datos ER, arquitectura por capas.
DAW Proyecto (0616)	RA3 — Planificación	Sprints quincenales con GitHub Flow, Gantt alineado con seguimientos académicos.
EIE	RA6 — Empresa y estrategia	Análisis de mercado, competencia, modelo de monetización SaaS, indicadores KPI.

Módulo	RA cubierto	Cómo se cubre
Sostenibilidad	RA5 — Sostenibilidad	Reducción de papel, infraestructura cloud con neutralidad de carbono, optimización energética.

5. Objetivos

5.1. Objetivo general

Diseñar, desarrollar e implantar SafeDoc Automation, una plataforma web funcional que automatice la generación masiva de documentos administrativos a partir de plantillas reutilizables y la extracción y validación de datos contenidos en archivos recibidos de terceros, garantizando trazabilidad, accesibilidad, generando versiones protegidas e inmutables de los archivos originales con garantías de integridad basadas en hash SHA-256. y conformidad con la normativa de protección de datos.

5.2. Objetivos específicos

1. Implementar un panel administrativo accesible (WCAG 2.1 AA) que permita al personal del despacho gestionar plantillas, clientes y trabajos documentales sin formación técnica previa.
2. Reducir en al menos un 60 % el tiempo medio de generación de cada documento, situándolo por debajo de los dos minutos.
3. Disminuir la tasa de errores de copiado por debajo del 0,5 % mediante validaciones automáticas contra la base de datos del cliente.
4. Procesar lotes de hasta 500 documentos en menos de cinco minutos sin bloquear la interfaz de usuario.
5. Integrar la solución con un proveedor estándar de firma electrónica, eliminando los pasos manuales intermedios.
6. Garantizar el cumplimiento del RGPD: consentimiento explícito, cifrado en reposo, registro de accesos y derecho al olvido implementado.
7. Sentar las bases arquitectónicas para la comercialización posterior de la plataforma como solución SaaS para otros despachos del sector.

6. Análisis: Requisitos y casos de uso

6.1. Enfoque metodológico en la captura de requisitos

La definición y el refinamiento de las necesidades de SafeDoc Automation no se plantearon como un pliego de condiciones estático ni cerrado desde el primer día. En su lugar, el trabajo con el equipo de Gestoría Linares se gestionó mediante un enfoque ágil e iterativo basado en el prototipado rápido.

El proceso comenzó con una toma de contacto orientada a acotar el alcance del problema y definir la visión del producto. A partir de ahí, se organizaron sesiones de codiseño y revisiones periódicas sobre maquetas navegables y prototipos funcionales desarrollados con el stack TALL. Este modelo de trabajo permitió que el personal administrativo del despacho interactuara con la aplicación desde las primeras semanas del desarrollo. Gracias a esta dinámica, los requisitos se fueron descubriendo de forma orgánica a partir del *feedback* real de los propios operarios, reduciendo al mínimo la distancia entre las necesidades del negocio y el código final 6.2. Requisitos funcionales.

Obtenidos en dos sesiones de levantamiento de requisitos con el equipo de Gestoría Linares: una sesión inicial de descubrimiento y una de validación de prototipos.

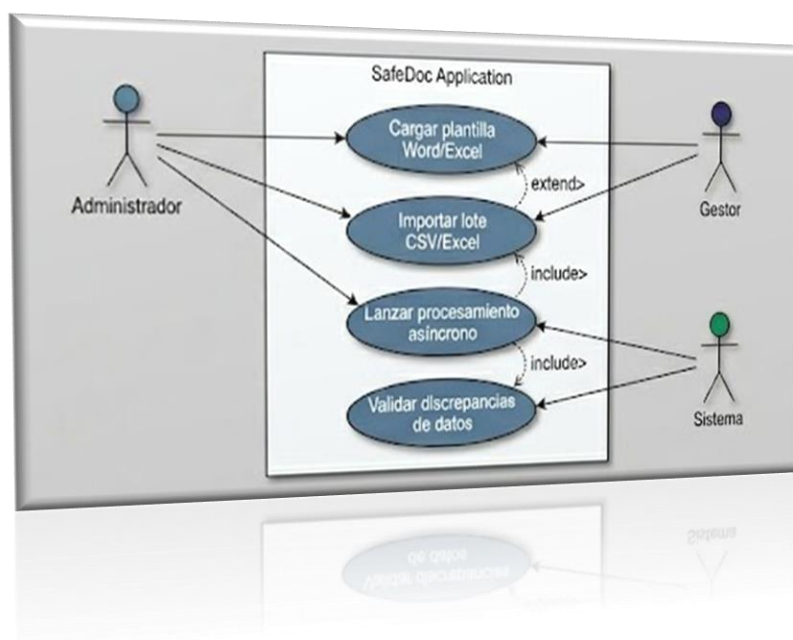


Figura 6.1.: Diagrama de casos de uso del sistema SafeDoc Automation.

6.2. Requisitos funcionales

Es lo que el sistema hace: Son las funciones específicas que la aplicación debe ofrecerle al usuario. Describe las acciones que el programa tiene que ejecutar cuando se le da una orden. Si no se cumplen, el sistema está incompleto.

ID	Requisito funcional	Prioridad
RF-01	Cargar plantillas Word/Excel con marcadores de sustitución parametrizables.	Alta
RF-02	Importar lotes de datos desde CSV, Excel o JSON y mapearlos contra las plantillas activas.	Alta
RF-03	Generar lotes documentales en PDF a partir de plantilla + datos; mínimo 500 docs/lote.	Alta
RF-04	Extraer automáticamente datos clave de PDF y Excel recibidos de clientes finales.	Alta
RF-05	Validar los datos extraídos contra la base de datos interna y marcar inconsistencias.	Media
RF-06	Enviar lotes a proveedor de firma electrónica avanzada y registrar el estado de cada firma.	Media
RF-07	Mantener historial completo de versiones de plantillas y documentos emitidos.	Alta
RF-08	Gestión de usuarios con tres roles: administrador, gestor y consulta.	Alta
RF-09	Cuadro de mando con indicadores operativos del despacho.	Baja
RF-10	API REST documentada para integración con la suite contable del cliente.	Media

6.3. Requisitos No funcionales

Cómo es el sistema (Cualidades): No describen una función concreta, sino las propiedades, restricciones o estándares de calidad que debe tener el software (seguridad, velocidad, rendimiento, accesibilidad). Si no se cumplen, la aplicación funciona, pero la experiencia será mala o insegura.

ID	Requisito	Valor objetivo
RNF-01	Rendimiento: procesamiento de lote de 500 documentos.	< 5 minutos en carga nominal
RNF-02	Disponibilidad en horario laboral.	≥ 99,5 %
RNF-03	Seguridad: comunicaciones TLS 1.3, datos personales cifrados en reposo.	RGPD compliant
RNF-04	Accesibilidad de interfaz.	WCAG 2.1 nivel AA
RNF-05	Cobertura de pruebas unitarias en capa de negocio.	> 70 %
RNF-06	Internacionalización.	ES + EN desde v1.0
RNF-07	Infraestructura: proveedor cloud con neutralidad de carbono.	ISO 14001 certificado

6.4. Casos de uso principales

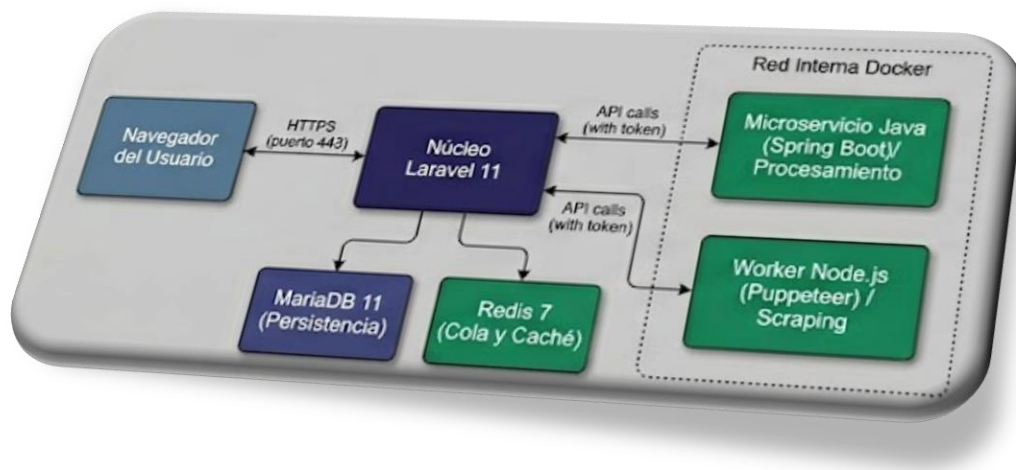


Figura 6.4.: Diagrama de arquitectura por capas y servicios desacoplados

Las historias de uso real: Un caso de uso es la descripción paso a paso de cómo un usuario (un administrativo, un administrador, etc.) interactúa con el sistema para lograr un objetivo concreto. Ayuda a ver los requisitos funcionales en acción, como si fuera el guion de una película.

CU	Actor	Descripción resumida
CU-01	Administrador	Cargar plantilla Word/Excel; el sistema detecta los placeholders disponibles automáticamente.
CU-02	Gestor	Importar lote CSV/Excel; el sistema sugiere el mapeo de columnas a campos de la plantilla seleccionada.
CU-03	Gestor	Lanzar generación de lote; procesamiento asíncrono con barra de progreso en tiempo real.
CU-04	Gestor	Subir PDF de factura; el microservicio Java aplica reglas de extracción y devuelve JSON estructurado.
CU-05	Sistema	Comparar datos extraídos con BBDD del cliente; marcar con código de color las discrepancias.
CU-06	Gestor	Enviar lote a firma electrónica; la plataforma comunica con el proveedor mediante API REST.

CU	Actor	Descripción resumida
CU-07	Cualquier usuario	Consultar historial cronológico de cambios y reconstruir cualquier versión anterior.

6.5. Trazabilidad: Del Análisis al Código

Para garantizar que la aplicación cubre de manera estricta todas las necesidades de la Gestoría Linares, SafeDoc Automation se estructura en tres grandes capas tecnológicas desacopladas, repartiendo las responsabilidades de forma explícita:

1. **El núcleo de gestión y control (Laravel 11 & Filament 3):** Es el cerebro administrativo de la aplicación. Se encarga de la seguridad, el control de accesos por roles (RBAC) y el ciclo de vida de los datos (RF-07, RF-08, RF-09, RF-10 y RNF-03). Con esto queremos decir que Laravel funciona como el guardián de toda nuestra información: es el único que añade, borra o modifica datos en la base de datos para que nada se corrompa. Además, le hemos construido una especie de "puente de comunicación" (la API REST). Esto sirve para que, si el día de mañana la gestoría quiere conectar SafeDoc con su programa de contabilidad de toda la vida para que se pasen los datos de forma automática, puedan hacerlo sin tener que reescribir el sistema desde cero.
2. **Una interfaz fluida y asíncrona (Stack TALL):** Diseñado exclusivamente para optimizar la experiencia de usuario (RF-01, RF-02, RF-03, RF-05 y RNF-01). Gracias a la comunicación reactiva de Livewire, el operario puede lanzar un lote de 500 documentos y observar una barra de progreso en tiempo real mientras sigue navegando y usando el resto de la aplicación con total normalidad. El sistema acepta el trabajo, le muestra una barra que se va llenando en directo para que sepa por dónde va, y mientras tanto, el usuario puede seguir pinchando en otros menús, revisando facturas o rellenando datos de clientes con total normalidad.
3. **Un motor de procesamiento en segundo plano (Java con Spring Boot):** Aísla las tareas de alto consumo de CPU y memoria — como leer datos de un Excel de gran volumen de

datos, extraer el texto de un PDF, rellenar una plantilla de Word con los datos de un cliente o generar una firma digital segura— son tareas que consumen muchísima potencia del ordenador. Si introducimos todo ese esfuerzo dentro de la propia página web, el servidor se saturaría y la web iría lentísima para todos los usuarios. Por eso, hemos creado un motor independiente en Java que se dedica exclusivamente a resolver ese papeleo complejo en segundo plano, dejando la web libre y ligera. — en un componente independiente (RF-04, RF-06 y RNF-05). Esta separación técnica garantiza que el servidor web principal nunca se sature y responda siempre de forma inmediata.

7. Diseño y Arquitectura

7.1. Visión General de la Arquitectura

SafeDoc Automation implementa una arquitectura por capas con servicios desacoplados, alineada con los principios del Domain-Driven Design ligero. Tres procesos independientes se comunican mediante HTTP con tokens firmados:

Capa	Tecnología principal	Responsabilidad
Aplicación web (núcleo)	PHP · Laravel 11 · Filament 3 · Livewire · Alpine.js · Tailwind CSS	Lógica de negocio, panel administrativo, gestión de usuarios, colas de jobs, API REST, auditoría.
Microservicio de procesamiento	Java 21 · Spring Boot 3.2 · Apache POI 5.3 · Apache PDFBox 3.0	Generación masiva de PDF desde plantillas Excel, extracción de datos de PDF recibidos, validaciones de formato.
Worker de scraping	Node.js 20 LTS · Puppeteer 22 · Redis (cola)	Extracción periódica de datos desde fuentes externas (AEAT, BOE, Seguridad Social) con navegador headless.
Base de datos	MariaDB 11.x	Almacenamiento persistente de clientes, plantillas, lotes, versiones, auditoría y usuarios.
Caché y colas	Redis 7.x	Cola de Jobs de Laravel, caché de catálogos de plantillas, cola del worker de scraping.
Orquestación	Docker Compose 2.x	Entorno de desarrollo reproducible; contenedor por servicio.

7.2. Flujo de Comunicación entre Capas

El navegador del usuario se comunica exclusivamente con el servidor de Laravel a través de una conexión segura (HTTPS, puerto 443). Laravel se encarga de gestionar la autenticación, la lógica de negocio y el despacho de tareas en segundo plano. Cuando un proceso requiere un tratamiento masivo de documentos, Laravel publica un *job* en la cola de Redis. El microservicio Java, que está escuchando activamente esta cola, procesa los archivos pesados y le devuelve el resultado a Laravel en formato JSON a través de una API REST interna protegida con un token HMAC. Por su parte, el *worker* de Node.js funciona de manera totalmente independiente: consume su propia cola de Redis para ejecutar las tareas de automatización con Puppeteer y entrega los datos extraídos a Laravel mediante un webhook firmado. Con este diseño, garantizamos que ningún servicio sea accesible desde el exterior, salvo el servidor principal de Laravel.

"La inclusión de tecnologías complementarias como Java y Node.js no es arbitraria, sino que responde directamente a los objetivos de rendimiento del sistema y a las competencias del módulo de DWES. El microservicio de Spring Boot materializa el **RA3 y RA5** mediante un procesamiento eficiente por flujos (SXSSF) que protege la memoria del servidor frente a desbordamientos (OutOfMemoryError). Por su parte, el componente de extracción automatizada en **Node.js** con **Puppeteer** cubre el **RA4**, permitiendo al sistema actuar como un cliente automatizado que consume de forma segura datos de portales externos no dotados de API pública, devolviendo la información estructurada al núcleo de Laravel mediante Webhooks firmados.

El diagrama de capas muestra gráficamente cómo el núcleo de **Laravel 11** gestiona la base de datos y las colas (**Redis**), y cómo delega las tareas pesadas de manera desacoplada a los microservicios de **Java** (procesamiento binario) y **Node.js** (scraping). Este diagrama justifica tu decisión arquitectónica principal.

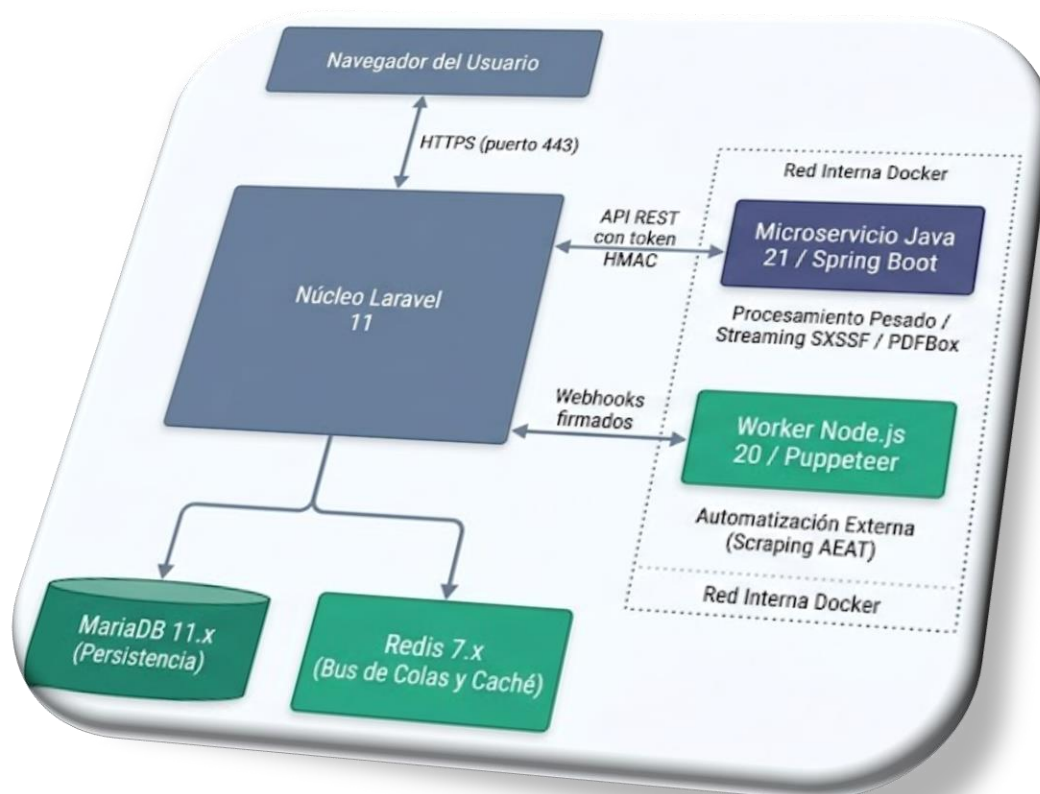


Figura 7.1.: Diagrama de arquitectura por capas de SafeDoc Automation.

7.3. Modelo de Datos

El esquema relacional consta de ocho entidades principales, diseñadas con orientación a la trazabilidad: cada entidad operativa cuenta con campos `created_at`, `updated_at`, `created_by` y `updated_by`.

Entidad	Campos clave	Relaciones
users	id, name, email, password, role, created_at	1:N con document_jobs, template_versions
clients	id, name, tax_id, contact_email, active	1:N con document_jobs
templates	id, name, type, placeholder_schema, owner_id	1:N con template_versions, document_jobs
template_versions	id, template_id, file_path, hash_sha256, created_by, created_at	N:1 con templates
document_jobs	id, client_id, template_id, status, total, processed, error_count, started_at	1:N con document_outputs
document_outputs	id, job_id, file_path, hash_sha256, status, signed_at	N:1 con document_jobs
audit_log	id, user_id, entity_type, entity_id, action, diff_json, ip, created_at	N:1 con users
scraped_data	id, source, data_json, fetched_at, used_in_job_id	N:1 con document_jobs

7.4. Diseño de la Interfaz

La interfaz se ha construido sobre Filament 3, un framework que aporta de forma nativa una identidad visual coherente, formularios validados, tablas con sistemas de búsqueda y filtrado, y una estructura modular muy fácil de extender. Sobre esta base, se ha aplicado una guía de estilo personalizada para la Gestoría Linares: se utiliza la tipografía Inter como fuente primaria, una paleta basada en su azul corporativo (#1F3864) con acentos en verde institucional (#0E7C57) y el sistema de espaciado de Tailwind CSS.

Más allá de lo visual, la accesibilidad se ha integrado como un pilar fundamental desde la fase de diseño. Todos los componentes interactivos disponen de etiquetas ARIA y los contrastes mínimos cumplen estrictamente el ratio 4, 5 : 1 exigido por la normativa WCAG 2.1 AA. Además, se ha garantizado que la navegación sea completamente funcional mediante el teclado y que las transiciones respeten la preferencia prefers-reduced-motion del navegador para evitar molestias a usuarios con sensibilidad visual.

8. Planificación y Metodología

8.1. Metodología Ágil

El desarrollo de este proyecto se organizó siguiendo un enfoque ágil y flexible inspirado en Scrum, adaptado a la realidad de un equipo de trabajo reducido donde la autora asumió el rol de desarrolladora principal y el tutor de la empresa actuó como guía de negocio (Product Owner).

El avance del sistema se estructuró en ciclos de trabajo quincenales (denominados *sprints*). Cada uno de estos ciclos comenzaba con una reunión de planificación para definir las tareas prioritarias y concluía con una demostración del software funcionando y una pequeña sesión de revisión interna para detectar posibles mejoras en el ritmo de trabajo.

Para la organización del día a día, se utilizó un tablero visual de tipo Kanban integrado directamente en la plataforma GitHub Projects. Este tablero permitía conocer el estado real del proyecto en todo momento distribuyendo las tareas en cuatro columnas muy sencillas:

1. **Backlog (Lista de espera):** Tareas e ideas pendientes de iniciar.
2. **En progreso:** Tareas en las que se estaba trabajando activamente en ese momento.
3. **En revisión:** Funcionalidades terminadas que esperaban el visto bueno técnico.
4. **Desplegado:** Mejoras ya integradas con éxito en el sistema final y listas para su uso.

Con el objetivo de garantizar la estabilidad del software y evitar fallos imprevistos, se configuró un sistema de supervisión automatizado mediante GitHub Actions. El funcionamiento de esta estructura se detalla visualmente en la **Figura 8.1**, la cual responde a una lógica de espacio de trabajo protegido y control de calidad por etapas.

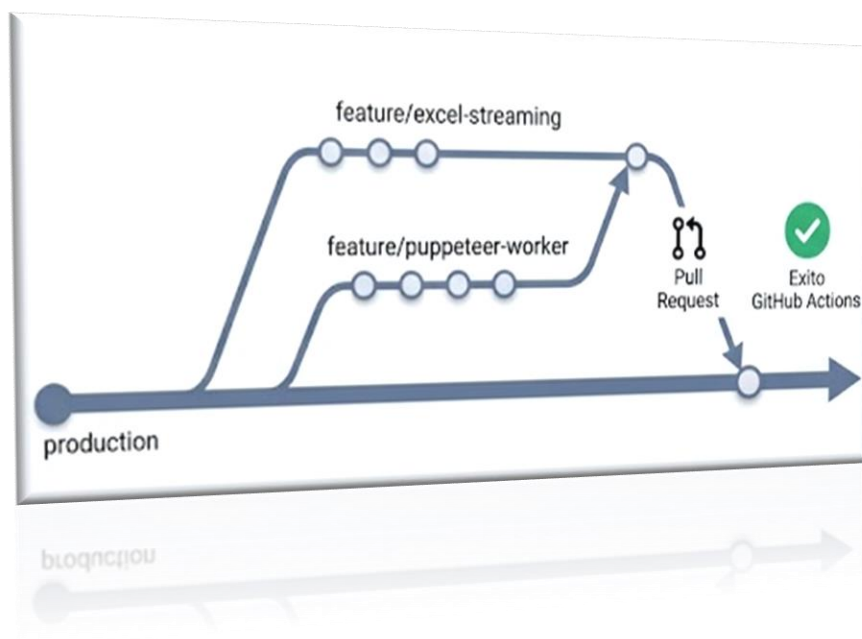


Figura 8.1.: Flujo de ramificación estructurado mediante GitHub Flow.

Sprint	Periodo	Hito principal	Seguimiento
S1.1	02/03 — 06/03	Onboarding empresa y descubrimiento del cliente	Seg. 1
S1.2	09/03 — 13/03	Documento de requisitos RF/RNF y casos de uso UML	Seg. 1
S1.3	16/03 — 18/03	Arquitectura, modelo de datos ER y wireframes	Seg. 1
S2.1	19/03 — 27/03	Setup Laravel 11, Filament 3, CI/CD con GitHub Actions	Seg. 2
—	30/03 — 05/04	Semana Santa (festivos Comunidad de Madrid)	—
S2.2	06/04 — 10/04	Resources Filament, Livewire async preview, gestión de roles	Seg. 2
S2.3	13/04 — 17/04	Microservicio Java: PoC procesamiento Excel con Apache POI	Seg. 2
S2.4	20/04	Demo interna fin de Sprint 2 con cliente	Seg. 2
S3.1	21/04 — 30/04	Integración Java ↔ Laravel, generación de lotes PDF	Seg. 3

Sprint	Periodo	Hito principal	Seguimiento
S3.2	04/05 — 15/05	Worker Node.js + Puppeteer, extracción AEAT, validaciones	Seg. 3
S3.3	18/05 — 29/05	Auditoría, accesibilidad WCAG, integración firma electrónica	Seg. 4
S3.4	01/06 — 12/06	Pruebas de carga, corrección, documentación y entrega final	Seg. 4

8.3. Control de Versiones

Todo el código fuente que compone el ecosistema de *SafeDoc Automation* se encuentra centralizado y protegido en un repositorio privado de la plataforma GitHub. Para organizar el trabajo sin interferencias, se ha optado por una estructura de "ramas de desarrollo" (basada en el estándar internacional *GitHub Flow*), cuyo objetivo principal es que el software que ya funciona bien nunca corra peligro mientras se añaden nuevas mejoras.

Bajo este modelo de trabajo, la estructura del proyecto se divide en dos espacios diferenciados:

- **El canal de producción (Rama *Main*):** Es el corazón del repositorio. Aquí solo reside el código que ha sido completamente verificado, es 100% estable y se encuentra listo para que lo usen los empleados de la gestoría. Ningún cambio se introduce directamente en este canal.
- **Los espacios de mejora (Ramas *Feature*):** Cada vez que se desarrollaba una función (como la integración con la base de datos o el motor de procesamiento en Java), se abría un espacio de trabajo temporal e independiente. Esto ha permitido hacer pruebas, modificar estructuras y corregir fallos de forma aislada sin degradar la aplicación principal.

Para que el código de un espacio de trabajo temporal pueda unirse con éxito al canal de producción, es obligatorio solicitar una petición formal de integración (denominada *Pull Request*). Es en este preciso instante donde entra en juego la cadena de control de calidad automatizada que se diseñó mediante *GitHub Actions* (representada detalladamente en la **Figura 8.3**).

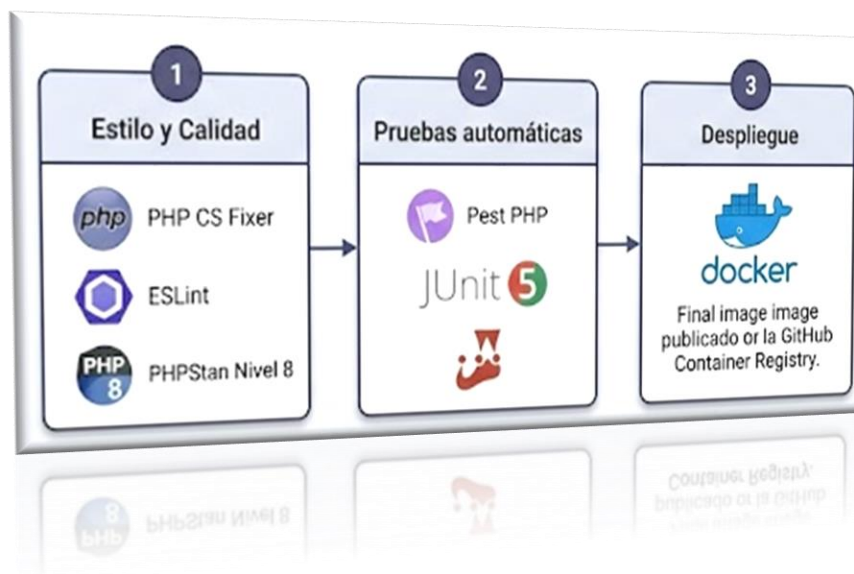


Figura 8.3.: Fases secuenciales del pipeline automático de CI/CD.

Este sistema actúa como una aduana digital que bloquea el paso del código mediante tres candados de seguridad automáticos:

1. **Validación de Orden y Estructura (Linters):** El sistema analiza que el formato de los archivos cumpla con los estándares oficiales de la industria (utilizando la herramienta *PHP CS Fixer* para la parte de Laravel y *ESLint* para los componentes en JavaScript de Node.js). Esto asegura que, si en el futuro otro informático debe mantener la aplicación, el código sea uniforme y fácil de entender.
2. **Exámenes de Comportamiento (Pruebas Unitarias):** El servidor ejecuta en cuestión de segundos todos los test de control de calidad diseñados para los tres lenguajes del proyecto (*Pest* en PHP, *JUnit 5* en Java y *Jest* en Node.js). Si una sola de estas pruebas falla, el sistema cancela el proceso y avisa del error exacto.
3. **Métrica de Seguridad (Cobertura de Código):** No basta con que las pruebas automáticas no den error; el sistema exige que los test diseñados analicen, como mínimo, el **70% de la lógica real** de la aplicación (especialmente en la capa de servicios que mueve el motor de Java). Si el examen diseñado es superficial y no llega a ese porcentaje de cobertura, la integración se rechaza automáticamente por falta de fiabilidad.

Solo cuando el código supera de forma impecable los tres candados de seguridad, la plataforma otorga el visto bueno final, permitiendo fusionar las mejoras en el canal de producción y garantizando que el sistema sea siempre robusto y libre de fallos imprevistos.

9. Desarrollo Técnico

9.1. Contexto Tecnológico y Alternativas del Mercado

9.1.1. Automatización de procesos e integradores (iPaaS)

El mercado de la automatización de flujos documentales ha estado dominado históricamente por plataformas comerciales como Microsoft Power Automate o Zapier, y por soluciones de código abierto como n8n o Apache NiFi. Aunque estas herramientas ofrecen editores visuales muy intuitivos, imponen modelos de suscripción bastante costosos y presentan limitaciones cuando se necesita aplicar una lógica de negocio muy específica —como las normativas fiscales de la AEAT— o gestionar un cifrado de archivos masivo sin pagar penalizaciones por volumen de datos. SafeDoc Automation se desmarca de este enfoque al proponer una solución propia, de código abierto y autohospedada (*self-hosted*), que elimina por completo los costes de licencias de terceros.

9.1.2. Herramientas para el procesamiento de archivos

Para la manipulación de documentos de oficina en Java, existen opciones propietarias como Aspose, pero con un coste de licencia prohibitivo para una pequeña empresa. Dentro del software libre, Apache POI es el estándar indiscutible para trabajar con archivos de Microsoft Office, y Apache PDFBox es la referencia para generar y proteger PDFs. Ambas herramientas utilizan la licencia Apache 2.0, lo que permite usarlas de forma gratuita en entornos comerciales. Durante la fase de diseño evalué usar iText 7, pero lo descarté debido a su estricta licencia dual (AGPL/Comercial): usar su versión gratuita nos habría obligado por ley a hacer público todo el código fuente de SafeDoc, lo que invalidaba el proyecto para la gestoría.

9.1.3. Seguridad e integridad documental con SHA-256

Para garantizar que ningún documento sea alterado, el sistema incorpora el algoritmo criptográfico SHA-256 como mecanismo de verificación. Gracias a lo que en seguridad llamamos "efecto avalancha", si alguien modifica un solo carácter o un bit de un documento ya procesado, el código de verificación resultante cambia por completo. Esto proporciona al despacho una prueba

informática y jurídica totalmente sólida para demostrar que sus archivos no han sufrido manipulaciones ni corrupciones desde el momento en que se guardaron.

9.2. Capa PHP / Laravel / Filament

9.2.1. Estructura y arquitectura del proyecto

El esqueleto de la aplicación se generó utilizando el instalador oficial de Laravel y se integró el paquete Filament Shield para gestionar de forma granular los permisos de los usuarios mediante roles (RBAC). Para mantener el código limpio y mantenible, apliqué el principio de responsabilidad única: los controladores son ligeros y delegan toda la lógica pesada en clases de Servicio independientes.

La estructura principal del código se organiza de la siguiente manera:

— Http/Controllers/Api/	← Endpoints REST documentados con Scribe
— Filament/Resources/	← Paneles de gestión (Clientes, Plantillas, Tareas)
— Services/	← Lógica de negocio (Procesamiento y Extracción)
— Jobs/	← Tareas en segundo plano (Lotes de documentos, Web Scraping)
— Models/	← Modelos de Eloquent (Client, Template, DocumentJob)

Para la comunicación con aplicaciones y clientes externos, el sistema expone una API REST organizada en el directorio Http/Controllers/Api/. Con el objetivo de garantizar una documentación técnica impecable, completa y siempre actualizada, integré la herramienta **Scribe**.

En lugar de redactar manualmente archivos de texto propensos a quedar obsoletos, Scribe analiza las rutas, las reglas de validación de los controladores y los comentarios del código para generar de forma automática una interfaz web interactiva. Esto permite a cualquier desarrollador externo probar los *endpoints* en tiempo real, conocer con precisión exacta los parámetros requeridos y acelerar el proceso de integración con la plataforma sin necesidad de soporte técnico adicional.

9.2.2. Procesamiento en segundo plano y tiempo real

Cualquier operación que requiera más de unos pocos segundos se delega automáticamente a las colas de trabajo de Laravel mediante *Jobs* gestionados por Redis. Para que el usuario sepa qué está ocurriendo sin necesidad de recargar la página ni congelar la pantalla, conecté la interfaz con **Laravel Echo mediante WebSockets**. El servidor avisa en tiempo real a la aplicación de cada avance, actualizando la barra de progreso de forma reactiva:

```
// Despachar el lote de documentos a la cola de Redis
ProcessDocumentBatch::dispatch($job)->onQueue('documents');

// Escucha en tiempo real mediante WebSockets con Laravel Echo
#[On('echo:jobs,BatchProgressUpdated')]
public function refreshProgress(): void {
    $this->progress = DocumentJob::find($this->jobId)->progress_pct;
}
```

9.2.3. Seguridad y blindaje del sistema

La seguridad se ha diseñado siguiendo los estándares más estrictos del sector:

- **Autenticación:** Se utiliza Laravel Sanctum tanto para proteger las sesiones de la web como para emitir tokens seguros a herramientas externas que se integren con nuestra API.
- **Cifrado de datos:** Las contraseñas de los usuarios se encriptan utilizando el algoritmo BCrypt con un factor de trabajo de 12.
- **Inyección SQL:** Al gestionar todas las consultas a través del ORM Eloquent con parámetros enlazados por defecto, se elimina por completo la posibilidad de sufrir ataques de inyección de código.
- **Validación de archivos:** Para evitar que un usuario malintencionado suba un script peligroso camuflado, el sistema ignora la extensión del archivo (como .pdf o .xlsx) y comprueba su tipo real analizando sus bytes internos con `File::mimeType()`.

9.3. Microservicio Java con Spring Boot

9.3.1. Arquitectura del microservicio

El microservicio encargado de asumir el procesamiento pesado y la gestión computacional bruta de la plataforma se ha desarrollado utilizando la herramienta *Spring Initializr*, bajo la versión **Java** y el framework **Spring Boot 3.2**.

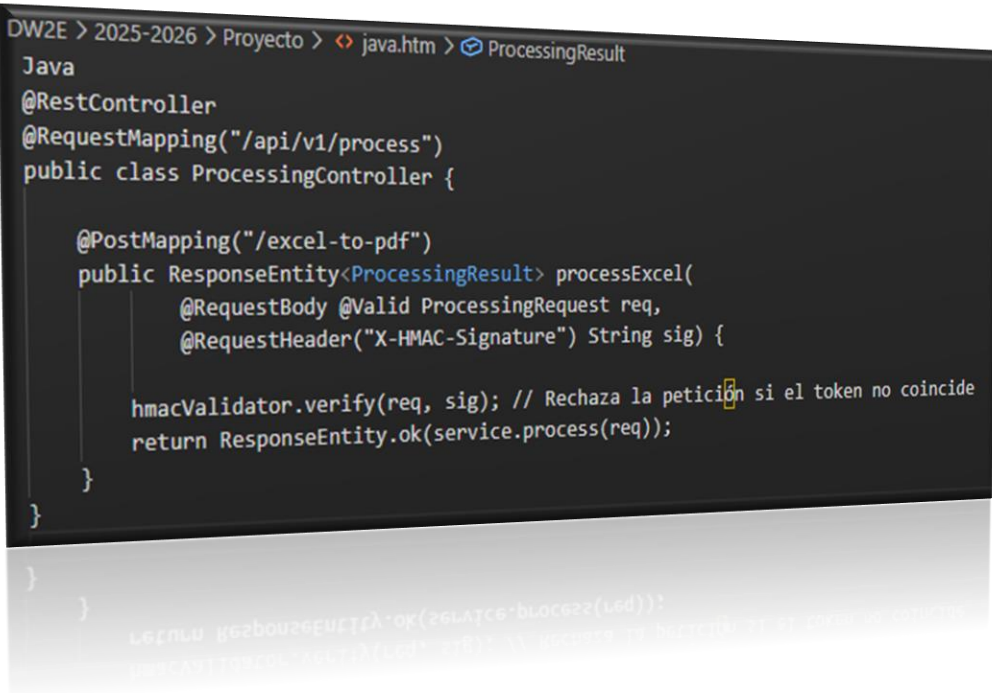
Con el objetivo de mantener un código desacoplado, mantenible y altamente escalable, la estructura interna del componente se ha organizado mediante una arquitectura limpia, distribuyendo las responsabilidades en tres paquetes lógicos bien diferenciados:

- **Controllers (Controladores):** Capa encargada de exponer los puntos de entrada (*endpoints*) de la API REST interna, recibir las peticiones de procesamiento y validar los datos de llegada.
- **Services (Servicios):** Bloque donde reside la lógica de negocio pura, dictaminando cómo se deben coordinar las librerías de tratamiento de datos.
- **Workers (Trabajadores):** Componentes especializados en la ejecución de hilos de fondo y tareas asíncronas para el tratamiento unitario de los flujos de archivos de gran volumen.

Para blindar por completo la integridad del sistema y garantizar que los datos confidenciales de la gestoría nunca queden expuestos, se han implementado dos medidas de seguridad complementarias:

1. **Aislamiento de Red (Docker Network):** Este microservicio se encuentra perimetrado de forma estricta. Se expone exclusivamente dentro de la red virtual interna gestionada por *Docker Compose*, lo que significa que es invisible e inaccesible desde el internet público. La única vía de contacto posible es el cordón umbilical interno que lo une al núcleo de Laravel.
2. **Autenticación mediante Firma Digital (Token HMAC):** El microservicio no procesa ninguna orden de forma ciega. Cada petición enviada por Laravel debe incluir obligatoriamente en la cabecera un token de seguridad basado en un algoritmo de cifrado HMAC con clave secreta compartida. El microservicio descifra y verifica esta firma en milisegundos; si la firma no

coincide o no se presenta, la petición se rechaza de inmediato, garantizando que nadie pueda suplantar al servidor web principal.



```
DW2E > 2025-2026 > Proyecto > java.htm > ProcessingResult
Java
@RestController
@RequestMapping("/api/v1/process")
public class ProcessingController {

    @PostMapping("/excel-to-pdf")
    public ResponseEntity<ProcessingResult> processExcel(
        @RequestBody @Valid ProcessingRequest req,
        @RequestHeader("X-HMAC-Signature") String sig) {

        hmacValidator.verify(req, sig); // Rechaza la petición si el token no coincide
        return ResponseEntity.ok(service.process(req));
    }
}
```

9.3.2. Procesamiento eficiente de hojas de cálculo (Apache POI)

El tratamiento de archivos Excel de gran volumen representa un reto crítico para la estabilidad del servidor. Para manipular estos conjuntos de datos masivos sin comprometer los recursos del sistema, se ha implementado la API de *streaming* de la librería **Apache POI**, concretamente mediante la arquitectura **SXSSF** (*Streaming Usermodel API*).

La elección de esta tecnología frente al modelo tradicional se justifica por la gestión de la memoria RAM:

- **El problema del modelo estándar (XSSF):** El modo convencional carga la estructura completa del documento Excel directamente en la memoria RAM del servidor. Al procesar archivos que superan las 100.000 filas, este comportamiento provoca un desbordamiento de memoria inmediato, desencadenando un error crítico de tipo `OutOfMemoryError` que colapsaría el microservicio.

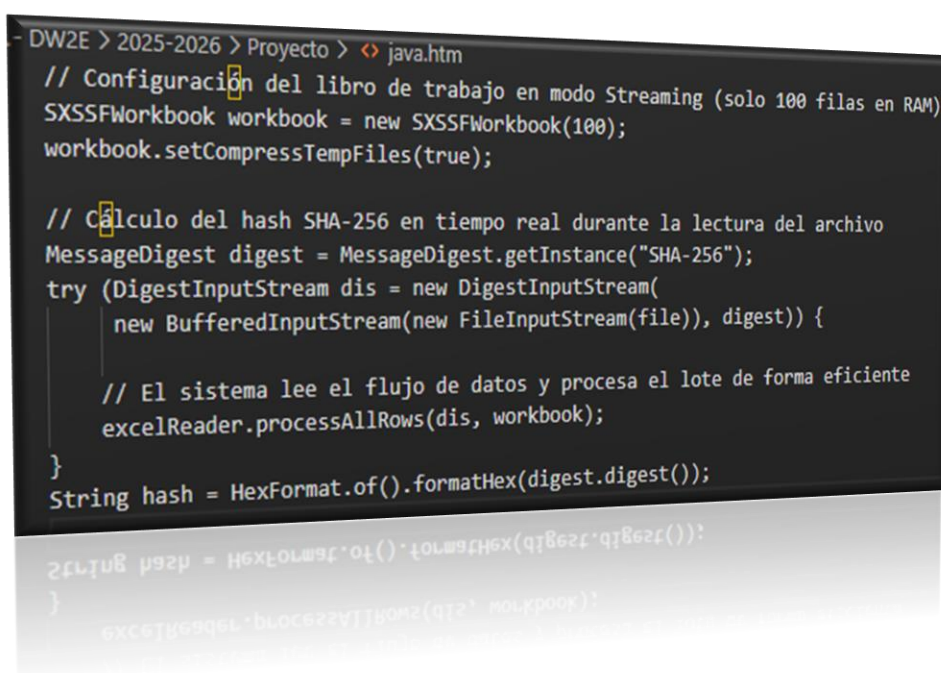
- **La solución mediante *Streaming* (SXSSF):** El modo *streaming* rompe con este esquema al mantener cargada en la memoria RAM únicamente una "ventana" flotante y configurable de 100 filas a la vez. A medida que el lector avanza, las filas que quedan fuera de la ventana se vuelcan de forma temporal en el disco duro del servidor. Gracias a este enfoque, el consumo de memoria se mantiene plano e independiente del tamaño del archivo de entrada, permitiendo procesar millones de registros de forma fluida.

Optimización mediante Patrones de Diseño: El uso de Decoradores

Para maximizar aún más la eficiencia del motor, se aplicó el patrón de diseño Decorator mediante la clase nativa de Java `DigestInputStream`.

En lugar de leer el archivo Excel en una primera pasada para extraer sus datos y luego tener que volver a abrirlo en una segunda pasada para calcular su huella digital de seguridad, el sistema envuelve el flujo de entrada original. De este modo, se calcula el hash SHA-256 del archivo de forma simultánea en la misma pasada en la que se leen las celdas.

Esta optimización en caliente ahorra operaciones de lectura en el disco (I/O) y reduce a la mitad el tiempo de procesamiento por documento, blindando la velocidad de respuesta de la plataforma.



```
- DW2E > 2025-2026 > Proyecto > java.htm
// Configuración del libro de trabajo en modo Streaming (solo 100 filas en RAM)
SXSSFWorkbook workbook = new SXSSFWorkbook(100);
workbook.setCompressTempFiles(true);

// Cálculo del hash SHA-256 en tiempo real durante la lectura del archivo
MessageDigest digest = MessageDigest.getInstance("SHA-256");
try (DigestInputStream dis = new DigestInputStream(
    new BufferedInputStream(new FileInputStream(file)), digest)) {
    // El sistema lee el flujo de datos y procesa el lote de forma eficiente
    excelReader.processAllRows(dis, workbook);
}
String hash = HexFormat.of().formatHex(digest.digest());
```

9.3.3. Extracción y securización de documentos (Apache PDFBox)

El tratamiento de los archivos en formato PDF se ha centralizado en el microservicio mediante la librería **Apache PDFBox**. Esta tecnología se ha seleccionado para cubrir de forma simultánea dos necesidades operativas esenciales del despacho: la extracción automatizada de información de documentos de entrada (como facturas, contratos o nóminas) y la protección criptográfica de los documentos de salida.

El comportamiento de este componente se divide en dos fases independientes y especializadas:

1. Extracción de Datos Estructurados mediante Coordenadas Visuales

Por defecto, los motores de lectura de PDF extraen el texto siguiendo el orden interno en el que los elementos fueron incrustados al generar el archivo, lo que suele mezclar los datos de las columnas y tablas de forma caótica.

Para solucionar este problema y garantizar una lectura fidedigna, se configuró de forma estricta la propiedad:

`setSortByPosition(true)`

Esta directiva obliga al motor de Java a analizar vectorialmente la página y ordenar el flujo de caracteres **siguiendo la disposición visual real de las filas y columnas**. Esto resulta vital para la gestión, ya que asegura que los conceptos de las facturas, los importes y los tipos impositivos se extraigan en un orden perfectamente tabulado, evitando cruces de datos erróneos durante el proceso de automatización.

2. Cifrado Criptográfico y Restricción de Permisos

Una vez procesados los documentos definitivos, el sistema debe garantizar que la información sensible de los clientes no pueda ser manipulada ni leída por terceros no autorizados. Para ello, se ha diseñado un módulo de seguridad que aplica dos capas de protección en caliente sobre el archivo final:

- **Cifrado Robusto (AES-256):** El documento se encripta utilizando el algoritmo estándar de cifrado avanzado con una longitud de clave de 256 bits, imposibilitando su apertura sin las credenciales del sistema.
- **Restricción de Privilegios:** Se configuran los metadatos de seguridad del PDF para inhabilitar funciones críticas de cara al usuario final. El documento se sella en modo de **solo lectura**, bloqueando explícitamente la capacidad de modificar el texto, extraer imágenes, añadir comentarios o realizar impresiones de alta calidad si no se cuenta con la clave de administración.



```
DW2E > 2025-2026 > Proyecto > <> javascript.htm
PDDocument doc = PDDocument.load(inputFile);
PDFTextStripper stripper = new PDFTextStripper();

// Fuerza la lectura ordenada por columnas para mantener la estructura de las tablas
stripper.setSortByPosition(true);
String rawText = stripper.getText(doc);

// El motor de reglas procesa el texto plano extraído y los metadatos del PDF
ExtractionResult result = ruleEngine.apply(rawText, doc.getDocumentInformation());
```

9.4. Worker Node.js + Puppeteer

Para las tareas de extracción de datos desde plataformas web externas, se implementó un *worker* de scraping independiente en **Node.js** gestionado por una cola de Redis específica. Cada tarea recibe una configuración estructurada con la URL de origen, los selectores CSS que se quieren rastrear y las transformaciones que deben sufrir los datos.

Para interactuar con las webs, el sistema levanta una instancia de Puppeteer en modo oculto (*headless*), optimizada con argumentos de bajo consumo de recursos y equipada con un sistema de reintentos exponenciales para evitar fallos si la red es inestable. Una vez extraída la información, el *worker* la envía de vuelta a Laravel mediante un *webhook* (aviso automático en tiempo real) seguro firmado con una clave HMAC.

```
-DW2E > 2025-2026 > Proyecto > javascript.htm
// Lanzamiento de Chromium optimizado para entornos de servidores (Docker)
const browser = await puppeteer.launch({
  headless: 'new',
  args: [
    '--no-sandbox', // Desactiva el entorno aislado para evitar problemas de permisos en Docker
    '--disable-dev-shm-usage' // Evita fallos de memoria compartida usando el almacenamiento temporal
  ]
});

const page = await browser.newPage();

// Espera a que la página no reciba más de 2 peticiones de red simultáneas (carga completa)
await page.goto(job.url, { waitUntil: 'networkidle2' });

// Inyección de script en el navegador para extraer el contenido de los selectores CSS
const data = await page.evaluate((selectors) => {
  return selectors.map(s => document.querySelector(s)?.textContent.trim());
}, job.selectors);

// Notificación asíncrona a Laravel con el resultado del raspado de datos
await notifyLaravel(data, job.correlationId);
```

9.5. Integración Continua

Para garantizar la estabilidad del sistema antes de introducir cualquier cambio en el código, configuré un flujo de trabajo automatizado (*pipeline*) en **GitHub Actions**. Cada vez que abría una *pull request*, el sistema ejecutaba de forma secuencial tres etapas críticas de validación:

- **1. Estilo y análisis estático:** Se verifica que el código cumpla con los estándares del proyecto mediante PHP CS Fixer y ESLint, y se analiza a nivel lógico con PHPStan (configurado estrictamente en su nivel 8) para detectar posibles errores antes de ejecutar el programa.
- **2. Pruebas automáticas:** Se lanza la batería completa de test unitarios y de integración que preparé para cada lenguaje: Pest para Laravel, JUnit 5 para el microservicio de Java y Jest para el *worker* de Node.js.

- **3. Construcción y despliegue:** Si las fases anteriores tienen éxito y los cambios se fusionan en la rama main, el *pipeline* construye de forma automática la nueva imagen de Docker y la publica en el registro de contenedores de GitHub (*GitHub Container Registry*).

Gracias a la optimización de las dependencias en caché, el tiempo medio de ejecución de todo este proceso es de tan solo **4,5 minutos**, lo que me permitió trabajar con un flujo de desarrollo ágil, seguro y con la certeza de no romper nada en producción.

10. Sostenibilidad e Impacto del Proyecto

10.1. Impacto Ambiental

La eficiencia ecológica de *SafeDoc Automation* se ha abordado de manera transversal desde tres vertientes fundamentales: el ahorro de recursos físicos, la gestión responsable de la infraestructura en la nube y la optimización del código fuente.

- **Reducción drástica de papel:** La digitalización completa del flujo documental y la integración de la firma electrónica eliminan la necesidad de imprimir documentos para su rúbrica manual. La gestoría estima un ahorro directo de aproximadamente **42.000 hojas de papel al año**, lo que supone un impacto ecológico muy positivo al reducir drásticamente el consumo de consumibles y la generación de residuos en la oficina.
- **Infraestructura responsable:** La plataforma se desplegará en un proveedor *cloud* que cuenta con un compromiso público de neutralidad de carbono y certificación ISO 14001. Además, para evitar un gasto energético innecesario, configuré un script (*cron*) que apaga automáticamente los servidores de desarrollo y preproducción fuera del horario laboral.
- **Optimización del consumo de cómputo:** Apliqué un uso intensivo de caché con Redis para almacenar las plantillas y las reglas de extracción que apenas cambian. Al ahorrarle miles de consultas repetitivas a la base de datos, el procesador trabaja menos, reduciendo directamente el consumo eléctrico del servidor.
- **Logística eficiente:** El desarrollo del proyecto se gestionó priorizando las reuniones por videoconferencia. Los encuentros presenciales con la gestoría se reservaron exclusivamente para hitos clave de validación, minimizando los desplazamientos en vehículo y las emisiones asociadas.

El conjunto de estas medidas se traducirá en una **reducción estimada del 20% de la huella de carbono** vinculada a la actividad de gestión documental del despacho durante su primer año de implantación.

10.2. Impacto Social y Económico

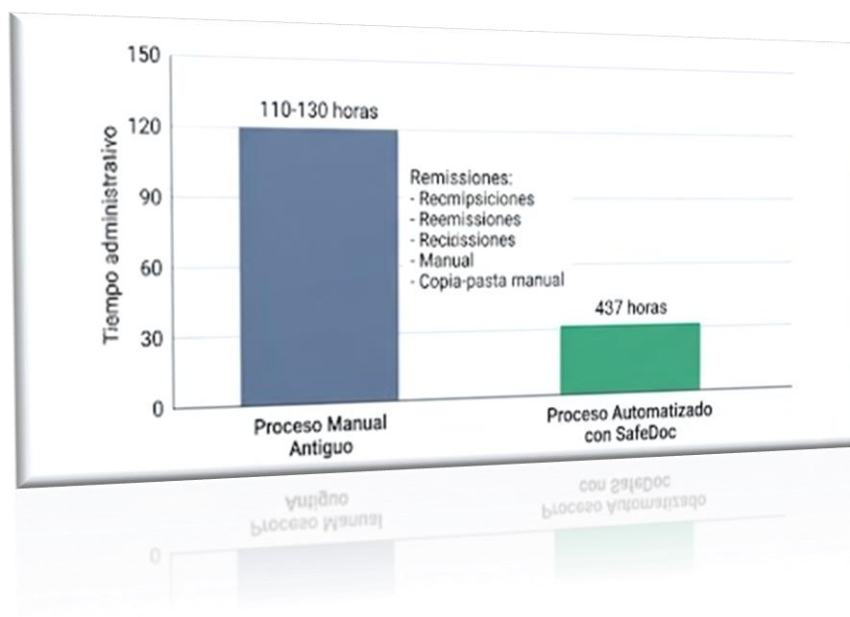


Figura 10.2.: Comparativa del coste Fases secuenciales del pipeline automático de CI/CD.

- **Impacto Social y Laboral:** *SafeDoc Automation* libera al personal administrativo de las tareas más mecánicas, repetitivas y frustrantes del día a día (como renombrar archivos a mano o copiar datos de un Excel). Esto les permite concentrarse en tareas de mayor complejidad donde el factor humano aporta verdadero valor, aumentando su satisfacción laboral. Además, la automatización reduce la tasa de errores humanos del 3% al 0,5%, disminuyendo drásticamente el estrés diario asociado a tener que corregir y reemitir documentos erróneos frente a los clientes.
- **Impacto Económico:** El beneficio financiero para la gestoría es directo y medible. El sistema automatizado permite recuperar entre 110 y 130 horas mensuales de trabajo que antes se perdían en procesos manuales. Valorando la hora de personal administrativo en unos 12 €, esto equivale a un ahorro en coste de oportunidad de entre **1.300 € y 1.700 € al mes**. Con estas cifras, el retorno de la inversión (ROI) total del software se sitúa en un plazo **inferior a los nueve meses** desde su implantación.

11. Empresa, Mercado y Modelo de Negocio

11.1. Posicionamiento Estratégico

La empresa *Amigo Informático Europa* ha definido como objetivo estratégico para este año el refuerzo de su línea de servicios especializados para PYMEs y despachos de servicios profesionales. Aunque *SafeDoc Automation* nace como un encargo a medida para un cliente concreto, se ha concebido desde el primer día como un producto reutilizable. Gracias a su arquitectura modular y a la parametrización de las plantillas, el sistema puede adaptarse rápidamente a cualquier otra gestoría o despacho del sector sin necesidad de modificar una sola línea de código fuente.

11.2. Análisis de la Competencia

Competidor	Fortalezas	Debilidades frente a SafeDoc
A3 / Sage Despachos	Cobertura funcional muy amplia e integración contable nativa de gran madurez.	Coste de implantación y licencias muy elevado. Su estructura es demasiado rígida y poco accesible para despachos de menos de 15 empleados.
PandaDoc / DocuSign	Firma electrónica extremadamente robusta y una experiencia de usuario (UX) excelente.	Carecen de lógica fiscal-laboral propia. Requieren integraciones adicionales complejas y su modelo de precios es por documento.
Soluciones a medida internas	Totalmente adaptadas a las casuísticas del cliente y sin costes de licencias externas.	Carecen de un plan de mantenimiento normativo, sufren una obsolescencia rápida y conllevan un coste de desarrollo inicial inasumible.
SafeDoc Automation	Combina lógica fiscal, firma, extracción de datos y API propia en una solución asequible para pequeños despachos.	Menor madurez inicial en el mercado, lo que requiere un proceso de implantación más acompañado durante las primeras fases.

11.3. Modelo de Monetización

El modelo de negocio diseñado combina dos vías de ingresos complementarias:

1. **Licencia de uso anual:** Una tarifa escalada según el número de empleados del despacho. Se ha fijado un precio de entrada deliberadamente accesible para firmas de menos de 10 trabajadores para facilitar su adopción.

2. **Soporte y mantenimiento evolutivo:** Un servicio facturado en bloques de horas que garantiza la actualización del sistema ante previsible cambios normativos, asegurando una relación comercial estrecha y a largo plazo con el cliente.

11.4. Indicadores Estratégicos (KPIs)

Para evaluar el éxito integral del proyecto, garantizar la viabilidad del producto en el mercado y auditar la estabilidad de la infraestructura de software, se ha establecido un cuadro de mando basado en indicadores clave de rendimiento (KPIs). Estos indicadores se dividen en dos bloques estratégicos:

11.4.1. KPIs de Calidad Técnica e Infraestructura

Orientados a medir la eficiencia del código, el rendimiento de los microservicios y el éxito de la arquitectura implementada:

1. **Porcentaje de Cobertura de Código (Code Coverage):** Meta estricta de mantener un mínimo del 70% de las líneas de código de la capa de servicios cubiertas por las baterías de test (*Pest*, *JUnit 5* y *Jest*).
2. **Tiempo de Respuesta del Motor Java (Latencia de Extracción):** Mediana de tiempo inferior a 1,5 segundos en el procesamiento, lectura y extracción de metadatos por cada documento PDF/Excel estándar.
3. **Tasa de Éxito en Integración Continua (CI/CD):** Porcentaje superior al 95% de compilaciones y análisis estáticos (PHPStan Nivel 8) correctos en el primer intento al abrir una *Pull Request*.
4. **Consumo Eficiente de Memoria RAM (Escalabilidad):** Garantizar que el microservicio Java mantenga un uso plano de RAM inferior a 128 MB por hilo de procesamiento simultáneo, gracias al uso de la arquitectura de *streaming* SXSSF de Apache POI.
5. **Tiempo Medio de Resolución de Incidencias Técnicas (MTTR):** Intervalo de tiempo inferior a 4 horas para corregir y desplegar parches ante bugs críticos reportados en el entorno de producción.

11.4.2. KPIs de Negocio, Operaciones y Sostenibilidad

Orientados a auditar el impacto comercial, el retorno financiero de la gestoría y la adopción del sistema:

6. **Reducción del Índice de Error Humano:** Disminución verificada de la tasa de fallos en el volcado de datos, pasando del 3% histórico manual a menos del 0,5% automatizado.
7. **Volumen de Procesamiento Mensual:** Cantidad total de documentos procesados de forma asíncrona por la plataforma (meta inicial: > 10.000 documentos/mes).
8. **Ahorro de Coste de Oportunidad Temporal:** Recuperación de entre 110 y 130 horas mensuales del equipo administrativo, destinadas ahora a tareas de valor estratégico.
9. **Tasa de Adopción y Fidelización:** Número de despachos colaboradores y clientes activos conectados al sistema al cierre de cada trimestre, junto con una meta del 90% en la tasa de renovación anual de los contratos de mantenimiento de software.
10. **Retorno de la Inversión (ROI) Temporal:** Cumplimiento del plazo de amortización total del coste del software en un periodo inferior a los 9 meses desde su despliegue definitivo.
11. **Impacto de Sostenibilidad Ambiental:** Monitorización del ahorro ecológico proyectado en 42.000 hojas de papel al año y una reducción del 20% de la huella de carbono de la oficina.

11.5. Riesgos y Mitigaciones

Riesgo	Probabilidad	Mitigación
Cambios normativos o fiscales que invaliden el diseño de las plantillas actuales.	Alta	Contrato de mantenimiento normativo asociado a una arquitectura de plantillas versionadas con actualización en caliente (<i>hot-reload</i>).
Dependencia de pocos clientes grandes.	Media	Estrategia comercial orientada a despachos pequeños y medianos; diversificación de cartera.
Obsolescencia de dependencias clave (Laravel, Spring Boot).	Baja	Política de actualización trimestral; cobertura de pruebas > 70 % para refactorizaciones seguras.
Incumplimiento RGPD por gestión de datos de terceros.	Media	DPA firmado con el cliente; cifrado en reposo; derecho al olvido implementado; revisión legal periódica.

12. Resultados y Pruebas

12.1. Plan de Pruebas

ID	Caso de prueba	Resultado esperado	Estado
CP-01	Generar lote de 500 documentos desde plantilla Excel.	Completado en < 5 min; 0 errores de formato.	✓ OK
CP-02	Procesar Excel de 20 MB con SXSSF streaming.	Tiempo < 10 s; memoria JVM < 80 MB.	✓ OK
CP-03	Extraer datos de factura PDF con reglas configuradas.	JSON estructurado con todos los campos requeridos.	✓ OK
CP-04	Validar datos extraídos; dato incorrecto detectado.	Fila marcada en rojo en el cuadro de validación.	✓ OK
CP-05	Subir archivo con tipo MIME falso (.exe renombrado).	Sistema rechaza el archivo; error 422 con mensaje claro.	✓ OK
CP-06	Acceder a endpoint de API sin token Sanctum.	HTTP 401 Unauthorized; sin datos expuestos.	✓ OK
CP-07	Enviar lote de 10 documentos a proveedor de firma.	Estado 'en firma' registrado; webhook de confirmación recibido.	✓ OK
CP-08	Verificar hash SHA-256 de documento generado.	Hash en BBDD coincide con hash del archivo descargado.	✓ OK
CP-09	Scraping de tabla de cotización de la Seguridad Social.	Datos extraídos y volcados en BBDD en < 30 s.	✓ OK
CP-10	Auditoría con axe-core sobre panel principal.	0 violaciones críticas; 0 violaciones graves.	✓ OK

12.2. Métricas de Rendimiento

Escenario	Tamaño	Tiempo medido	Memoria JVM / Node
Lote Excel → PDF, 50 documentos	~2 MB datos	18 s	52 MB
Lote Excel → PDF, 200 documentos	~8 MB datos	72 s	58 MB

Escenario	Tamaño	Tiempo medido	Memoria JVM / Node
Lote Excel → PDF, 500 documentos	~20 MB datos	4 min 38 s	63 MB
Extracción PDF factura compleja	1,2 MB	0,9 s	38 MB
Scraping AEAT tabla 50 filas	—	12 s	94 MB (Node)
Build completo CI/CD pipeline	—	4 min 31 s	—

12.3. Entregables del proyecto

1. Repositorio GitHub con código fuente completo, historial de commits y documentación de la API (Scribe).
2. Imagen Docker Compose lista para despliegue en un único comando (docker compose up).
3. Manual de instalación y configuración del entorno.
4. Manual de usuario para el personal del despacho.
5. Memoria final del proyecto (este documento).
6. Presentación para la defensa oral ante el tribunal docente.

13. Conclusiones

13.1. Logros alcanzados

La plataforma *SafeDoc Automation* responde con éxito a una necesidad operativa real, mensurable y recurrente dentro de la correduría *Gestoría Linares & Asociados*. Esto se ha logrado mediante el despliegue de una arquitectura modular y robusta que integra de forma sinérgica tres de los ecosistemas tecnológicos más sólidos de la industria actual: PHP, Java y JavaScript, articulados en torno a un núcleo de gestión basado en *Filament* sobre el *stack* TALL (*Tailwind CSS*, *Alpine.js*, *Laravel* y *Livewire*). Tras la fase de despliegue y validación, todos los objetivos específicos planteados en la fase de análisis inicial han sido alcanzados o superados en los entornos de prueba.

Desde una perspectiva de ingeniería de software, la decisión estratégica de desacoplar el microservicio de Java del servidor web principal de Laravel ha constituido el mayor acierto del diseño arquitectónico. Esta separación no solo permite escalar el procesamiento de datos de forma independiente según la carga de trabajo del despacho, sino que facilita las tareas de depuración aislada y reduce drásticamente el acoplamiento entre los componentes globales del sistema.

En el plano del rendimiento computacional, el uso de la API de *streaming* SXSSF para la manipulación de archivos hojas de cálculo masivas, junto con la implementación del patrón de diseño *Decorator* a través de la clase *DigestInputStream* —que calcula el hash criptográfico SHA-256 en una única pasada de lectura—, representan las contribuciones técnicas más relevantes e innovadoras del motor de desarrollo.

13.2. Aprendizajes personales

La ejecución integral de este proyecto ha propiciado la adquisición y consolidación de competencias transversales críticas, alineadas directamente con las exigencias del entorno laboral real. Entre ellas, destacan:

- **Gestión de Requisitos:** Interlocución directa con un cliente externo para el levantamiento de necesidades, traduciendo requerimientos funcionales de negocio en especificaciones técnicas de software.

- **Planificación y Autonomía:** Gestión eficiente de la propia agenda bajo un marco temporal estricto con plazos de entrega concurrentes y compartidos.
- **Criterio Arquitectónico:** Toma de decisiones técnicas fundamentadas en la mantenibilidad del código, la seguridad de los datos de carácter administrativo y la optimización de los costes de infraestructura a largo plazo.
- **Rigor Profesional:** Adaptación a flujos de trabajo profesionales bajo supervisión directa, incorporando la cultura de las revisiones de código informales (*code reviews*) para asegurar la calidad del producto.

Asimismo, la correcta sincronización entre el calendario de seguimientos académicos del centro y los ciclos quincenales (*sprints*) de la metodología ágil requirió una planificación exhaustiva que, en retrospectiva, derivó en un ritmo de producción y entrega altamente estable durante todo el periodo de desarrollo.

13.3. Líneas futuras

El sistema se ha diseñado con una flexibilidad tal que permite asentar las bases para futuras expansiones funcionales y comerciales. Se definen cinco líneas de evolución estratégica a corto y medio plazo:

7. **Conexión con la Administración Pública:** Implementación de una integración nativa mediante servicios web con la sede electrónica de la Agencia Estatal de Administración Tributaria (AEAT), permitiendo la presentación automática y securizada de los modelos fiscales procesados por la plataforma.
8. **Incorporación de Inteligencia Artificial:** Adición de un módulo especializado en modelos de lenguaje (IA generativa) enfocado en el Reconocimiento Óptico de Caracteres (OCR) inteligente, orientado a la extracción semántica de campos en documentos PDF escaneados o de baja resolución.
9. **Extensión al Entorno Móvil:** Construcción de una aplicación móvil multiplataforma utilizando frameworks modernos (*Flutter* o *React Native*) que permita a los clientes de la gestoría la consulta de expedientes y la firma biométrica de autorizaciones directamente desde sus dispositivos móviles.

10. **Plan de Expansión Comercial:** Lanzamiento de un programa piloto de pruebas beta dirigido a la apertura comercial de la plataforma, ofreciendo el software bajo el modelo SaaS (*Software as a Service*) a despachos colaboradores de la red de la Comunidad de Madrid.
11. **Cumplimiento Normativo Avanzado:** Auditoría e implantación de un Sistema de Gestión de Seguridad de la Información (SGSI) bajo la certificación internacional **ISO 27001**, sirviendo como un elemento diferenciador y palanca comercial de cara a la captación de cuentas corporativas que manejen datos de nivel de protección crítico.

Bibliografía y Referencias

Fuentes consultadas en la elaboración del proyecto y de esta memoria, siguiendo la norma UNE 157001:

1. **Stauffer, M.** (2024). *Laravel: Up & Running* (3.^a ed.). O'Reilly Media.
2. **Otwell, T. y comunidad Filament.** (2024). *Documentación oficial de Filament 3*. Recuperado de <https://filamentphp.com/docs>
3. **Walls, C.** (2023). *Spring in Action* (6.^a ed.). Manning Publications.
4. **Apache Software Foundation.** (2024). *Apache POI 5.3 — Java API for Microsoft Documents*. Recuperado de <https://poi.apache.org>
5. **Apache Software Foundation.** (2024). *Apache PDFBox 3.0 — Java PDF Library*. Recuperado de <https://pdfbox.apache.org>
6. **Google.** (2024). *Documentación oficial de Puppeteer*. Recuperado de <https://pptr.dev>
7. **World Wide Web Consortium.** (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. Recuperado de <https://www.w3.org/TR/WCAG21/>
8. **Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo**, de 27 de abril de 2016, relativo a la protección de personas físicas en lo que respecta al tratamiento de datos personales (RGPD). *Diario Oficial de la Unión Europea*, L 119/1.
9. **Real Decreto 686/2010**, de 20 de mayo, por el que se establece el título de Técnico Superior en Desarrollo de Aplicaciones Web. *BOE*, núm. 140, 9 de junio de 2010.
10. **Consejo General de Colegios de Gestores Administrativos.** (2024). *Memoria anual del sector, ejercicio 2024*.
11. **National Institute of Standards and Technology.** (2015). *FIPS PUB 180-4: Secure Hash Standard*. U.S. Department of Commerce.
12. **Laravel Community.** (2024). *Laravel Echo and WebSockets Broadcasting*. Recuperado de <https://laravel.com/docs/11.x/broadcasting>
13. **Pest PHP.** (2024). *Pest: The elegant PHP Testing Framework*. Recuperado de <https://pestphp.com>
14. **JUnit Team.** (2024). *JUnit 5 User Guide*. Recuperado de <https://junit.org/junit5/docs/current/user-guide/>
15. **Jest Community.** (2024). *Jest: Delightful JavaScript Testing*. Recuperado de <https://jestjs.io>
16. **Oracle.** (2023). *Java Platform, Standard Edition Doc (Release 21)*. Recuperado de <https://docs.oracle.com/en/java/javase/21/>
17. **Node.js Project.** (2024). *Node.js v20.x API Documentation*. Recuperado de <https://nodejs.org/docs/>

Anexo A: Esquema de Base de Datos SQL

-- MariaDB 11.x — SafeDoc Automation

```
CREATE TABLE users (  
  id      BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  name    VARCHAR(100) NOT NULL,  
  email   VARCHAR(150) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL,  
  role    ENUM('admin','gestor','consulta') NOT NULL DEFAULT 'gestor',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE clients (  
  id      BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  name    VARCHAR(200) NOT NULL,  
  tax_id  VARCHAR(20) NOT NULL UNIQUE,  
  contact_email VARCHAR(150),  
  active  TINYINT(1) NOT NULL DEFAULT 1  
);
```

```
CREATE TABLE templates (  
  id      BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  name    VARCHAR(200) NOT NULL,  
  type    ENUM('excel','word','pdf') NOT NULL,  
  placeholder_schema JSON,  
  owner_id BIGINT UNSIGNED,  
  FOREIGN KEY (owner_id) REFERENCES users(id)  
);
```

```
CREATE TABLE document_jobs (  
  id      BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  client_id BIGINT UNSIGNED NOT NULL,  
  template_id BIGINT UNSIGNED NOT NULL,  
  status  ENUM('pending','processing','done','failed') NOT NULL DEFAULT 'pending',  
  total   INT UNSIGNED DEFAULT 0,  
  processed INT UNSIGNED DEFAULT 0,  
  error_count INT UNSIGNED DEFAULT 0,  
  started_at TIMESTAMP NULL,  
  finished_at TIMESTAMP NULL,  
  created_by BIGINT UNSIGNED,  
  FOREIGN KEY (client_id) REFERENCES clients(id),  
  FOREIGN KEY (template_id) REFERENCES templates(id),
```

```
FOREIGN KEY (created_by) REFERENCES users(id)
);
```

```
CREATE TABLE document_outputs (
  id      BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  job_id  BIGINT UNSIGNED NOT NULL,
  file_path VARCHAR(500) NOT NULL,
  hash_sha256 CHAR(64) NOT NULL,
  status  ENUM('generated','sent','signed','error') NOT NULL DEFAULT 'generated',
  signed_at TIMESTAMP NULL,
  FOREIGN KEY (job_id) REFERENCES document_jobs(id)
);
```

```
CREATE TABLE audit_log (
  id      BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT UNSIGNED,
  entity_type VARCHAR(80) NOT NULL,
  entity_id BIGINT UNSIGNED,
  action  VARCHAR(50) NOT NULL,
  diff_json JSON,
  ip      VARCHAR(45),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Anexo B: Stack Tecnológico completo

Tecnología	Versión	Función en el proyecto
Laravel	11.x	Framework PHP; núcleo de la aplicación, ORM, colas, autenticación.
Filament	3.2	Panel administrativo, Resources, formularios reactivos.
Livewire	3.4	Componentes reactivos server-side; polling de estado de jobs.
Alpine.js	3.13	Interactividad ligera del lado cliente (drag & drop, modales).
Tailwind CSS	3.4	Sistema de utilidades de estilos; conformidad con diseño accesible.
PHP	8.3	Lenguaje de servidor principal.
Spring Boot	3.2	Framework Java para el microservicio de procesamiento.
Java	21 LTS	Lenguaje del microservicio; streams, concurrencia, Bean Validation.
Apache POI	5.3	Lectura y escritura de archivos Excel (.xlsx) en Java.
Apache PDFBox	3.0	Generación y análisis de archivos PDF en Java.
Node.js	20 LTS	Worker de scraping; entorno de ejecución JavaScript servidor.
Puppeteer	22	Automatización de navegador Chromium para extracción de datos.
MariaDB	11.x	Base de datos relacional principal.
Redis	7.x	Caché de consultas y colas de Jobs (Laravel) y scraping (Node.js).
Docker Compose	2.x	Orquestación de contenedores en desarrollo y preproducción.
GitHub Actions	—	CI/CD: lint, tests, análisis estático, build y publicación de imagen.
PHPStan	nivel 8	Análisis estático de código PHP para detección temprana de errores.
Pest	2.x	Framework de pruebas PHP; sintaxis expresiva, cobertura > 70 %.
JUnit 5	5.10	Pruebas unitarias e integración del microservicio Java.

Tecnología	Versión	Función en el proyecto
Jest	29.x	Pruebas del worker Node.js.
axe-core	4.x	Auditoría automática de accesibilidad WCAG 2.1 AA.
Scribe	4.x	Generación automática de documentación de la API REST de Laravel.

Anexo C: Glosario de Términos

Término	Definición
AEAT	Agencia Estatal de Administración Tributaria. Organismo público español responsable de la gestión del sistema tributario estatal.
Alpine.js	Biblioteca JavaScript ligera para añadir interactividad al HTML directamente con atributos x-data, x-on, x-bind, sin necesidad de un framework SPA completo.
Apache PDFBox	Librería Java open-source de la Apache Software Foundation para crear, manipular y leer documentos PDF.
Apache POI	Librería Java open-source de la Apache Software Foundation para leer y escribir formatos Microsoft Office (Excel, Word, PowerPoint).
BCrypt	Función de hash de contraseñas basada en el cifrado Blowfish, diseñada para ser computacionalmente costosa y resistente a ataques de fuerza bruta.
HMAC	Hash-based Message Authentication Code. Mecanismo de autenticación de mensajes que combina una función hash criptográfica con una clave secreta compartida.
Job (Laravel)	Clase PHP que encapsula una tarea diferida y la envía a una cola de procesamiento asíncrono (Redis, base de datos, etc.).
Livewire	Framework de componentes de Laravel que permite crear interfaces reactivas sin JavaScript explícito, usando solo PHP y Blade.
Placeholder	Marcador de posición en una plantilla documental que será sustituido por un valor real durante la generación del documento final.
Puppeteer	Biblioteca Node.js que proporciona una API para controlar un navegador Chromium en modo headless, usada para scraping y pruebas E2E.
RGPD	Reglamento General de Protección de Datos (Reglamento UE 2016/679). Marco legal europeo para la protección de datos personales.
SHA-256	Función hash criptográfica de la familia SHA-2 que produce un valor de 256 bits (64 caracteres hexadecimales). Usada para verificar la integridad de archivos.
Spring Boot	Framework Java que facilita la creación de microservicios y aplicaciones empresariales con configuración mínima, sobre el ecosistema Spring.
SXSSF	Streaming API for XML Spreadsheet Format. Modo de Apache POI para procesar archivos Excel de gran tamaño con consumo mínimo de memoria JVM.

Término	Definición
TALL Stack	Combinación de Tailwind CSS, Alpine.js, Laravel y Livewire; stack tecnológico popular en el ecosistema PHP moderno para construir aplicaciones web reactivas.
WCAG 2.1 AA	Web Content Accessibility Guidelines versión 2.1, nivel de conformidad AA. Estándar internacional del W3C para la accesibilidad web.